

BẢN ĐẶC TẢ KỸ THUẬT DỊCH CHUYỂN HỆ THỐNG

Chuyển đổi Công cụ CreateLashingHole từ VBA sang AutoCAD .NET API (C# / WPF)

Tài liệu này định nghĩa chi tiết kiến trúc cấu trúc mã nguồn, mô hình dữ liệu và các dịch vụ hình học phục vụ cho việc chuyển đổi công cụ **CreateLashingHole** (từ file gốc **CreateLashingHole.dvb**) sang nền tảng **AutoCAD .NET API** sử dụng mô hình **MVVM** tích hợp trong kiến trúc dự án hiện tại (Repository **SDS**).

1. Sơ Đồ Đối Chiếu Thành Phần (Mapping Components)

Để đảm bảo nguyên tắc Tách biệt các mối quan tâm (Separation of Concerns - SoC), các module trong mã nguồn VBA cũ sẽ được ánh xạ sang cấu trúc mới như sau:

Thành phần VBA gốc	Thành phần .NET mục tiêu	Vị trí thư mục trong dự án SDS
Khởi tạo biến phân tán	LashingHoleModels.cs (Dữ liệu POCO)	Models/
frmLashingInput.frm/.frx	LashingHolePalette.xaml (WPF Control)	UI/
mod_LashingHoleInterference.bas	CollisionEngineService.cs (Tính toán trong RAM)	Services/
mod_MainLashingHole_V3_5.bas	GridGenerationService.cs & BlockPackingService.cs	Services/
ModInputHelper_V3.bas	AutoCADGeometryHelper.cs (Tương tác DB CAD)	Utilities/
Lệnh Macro gốc	LashingHoleCommands.cs (Lệnh CFS_CreateLashingHole)	Commands/

2. Mô Hình Dữ Liệu Khởi Tạo (📁 Models/LashingHoleModels.cs)

Xây dựng các lớp dữ liệu độc lập, kiểm soát kiểu dữ liệu nghiêm ngặt thay thế cho biến **Variant** động của VBA:

```
namespace SDS.Models
{
    public enum LashingLocationMode
    {
        PortSide,    // Điểm neo P1 ở góc Dưới - Trái
        StarBoard,   // Điểm neo P1 ở góc Trên - Trái
        Center       // Tự động rải lưới bắt đầu từ tâm hình học
    }

    public enum LashingCollisionType
    {
        None,
        Vertical,
        Horizontal,
        Complex
    }

    public class LashingInputParams
    {
        public double HoleDiameter { get; set; } = 55.0;
        public double ClearanceRadius { get; set; } = 75.0;
        public double OffsetX { get; set; } = 150.0;
        public double OffsetY { get; set; } = 150.0;
        public double SpacingX { get; set; } = 500.0;
        public double SpacingY { get; set; } = 500.0;
        public string PanelName { get; set; } = "PNL_01";
        public LashingLocationMode LocationMode { get; set; } =
LashingLocationMode.PortSide;
        public bool IsAutomaticMode { get; set; } = false;
        public bool IsCheckAdjacent { get; set; } = false;
    }
}
```

3. Lớp Xử Lý Thuật Toán Hình Học (📁 Services/)

A. CollisionEngineService.cs (Bộ Máy Tính Toán Va Chạm)

Thực hiện toàn bộ các phép toán giao cắt trực tiếp trong RAM thông qua các đối tượng ảo (In-Memory Geometries) trước khi lưu vào cơ sở dữ liệu CAD:

- `ClassifyCollision(Circle testCircle, Entity structure)`: Gọi phương thức `testCircle.IntersectWith()` bản bản xứ. Khi phát hiện từ 2 giao điểm trở lên, tính toán hiệu số DeltaX, DeltaY của cặp điểm đầu tiên. Phân loại cấu kiện va chạm dọc nếu `DeltaY > DeltaX * 5.0` hoặc ngang nếu ngược lại.
- `IsPointInsidePolyline(Point3d pt, Polyline boundary)`: Loại bỏ hoàn toàn thuật toán bắn tia thủ công phức tạp của VBA. Sử dụng đối tượng `Region.CreateFromCurves()` và toán tử Boolean Intersect để xác định điểm nằm trong đa tuyến khép kín một cách tuyệt đối chính xác.
- `FindSafeShiftPoint(...)`: Kế thừa logic tìm điểm an toàn của hàm `FindBestSafeComplexShift_Local`. Chạy tịnh tiến các vector 8 hướng với bước dịch `5mm` ảo trong bộ nhớ nhằm tìm ra tọa độ mới mà không làm phát sinh thực thể rác trên bản vẽ.

B. GridGenerationService.cs (Bộ Rải Lưới Điểm)

- **Smart Bounding Box Logic**: Đọc danh sách các đỉnh của Polyline đường biên (Layer `Mechanical-AM_0`). Định nghĩa chính xác tọa độ điểm P1 và P2 dựa theo cấu hình `LocationMode`.
- **Gap Handling Logic**: Kiểm tra khoảng cách còn lại từ lỗ cuối cùng tới đường biên. Nếu khoảng cách nhỏ hơn ngưỡng `MIN_EDGE_DISTANCE_FOR_GAP = 200`, tự động kích hoạt thuật toán điều chỉnh vị trí điểm kề cuối (`$L_{i-1}$`) và khóa tọa độ an toàn.

4. Thiết Kế Giao Diện WPF Hiện Đại (📁 UI/LashingHolePalette.xaml)

Thay thế giao diện MS Forms cũ bằng cấu trúc WPF đồng bộ giao diện Dark Theme chuyên nghiệp:

```

<UserControl x:Class="SDS.UI.LashingHolePalette" Background="#2D2D2D">
    <ScrollView>
        <StackPanel Margin="10">
            <TextBlock Text="LASHING HOLE PARAMETERS" Foreground="#F1F1F1"
FontWeight="Bold"/>
            <Label Content="Hole Diameter (D):" Foreground="#9CDCFE"/>
            <TextBox Name="TxtHoleDiameter" Text="55" Background="#333333"
Foreground="White"/>
            <Label Content="Clearance Radius (R):" Foreground="#9CDCFE"/>
            <TextBox Name="TxtClearanceRadius" Text="75" Background="#333333"
Foreground="White"/>
            <Button Content="CREATE LASHING HOLES" Click="CreateBtn_Click"
Background="#007ACC" Foreground="White" FontWeight="Bold" />
        </StackPanel>
    </ScrollView>
</UserControl>

```

5. Đăng Ký Lệnh Hệ Thống (Commands/LashingHoleCommands.cs)

Đảm bảo an toàn giao dịch tối đa bằng cơ chế quản lý **Transaction**. Mọi thao tác lỗi hoặc người dùng nhấn hủy lệnh bằng nút **ESC** sẽ tự động thực hiện **Abort()** đưa bản vẽ về nguyên trạng sạch sẽ:

```

using System;
using Autodesk.AutoCAD.Runtime;
using Autodesk.AutoCAD.ApplicationServices;
using Autodesk.AutoCAD.DatabaseServices;
using Autodesk.AutoCAD.EditorInput;

namespace SDS.UI.Commands
{
    public class LashingHoleCommands
    {
        [CommandMethod("CFS_CreateLashingHole")]
        public void ExecuteCreateLashingHole()
        {
            Document doc = Application.DocumentManager.MdiActiveDocument;
            if (doc == null) return;
            Editor ed = doc.Editor;
            Database db = doc.Database;

            Autodesk.AutoCAD.Internal.Utils.SetFocusToDwgView();

            // Khởi tạo Transaction an toàn
            using (Transaction tr = db.TransactionManager.StartTransaction())
            {
                try
                {
                    // Logic xử lý rải lưới lỗ và kiểm tra cấu kiện...
                    tr.Commit();
                }
                catch (System.Exception ex)
                {
                    ed.WriteMessage("\nLỗi hệ thống, tự động Rollback: " +
ex.Message);
                    tr.Abort();
                }
            }
        }
    }
}

```

Lưu ý kỹ thuật khi tự triển khai: Khi chuyển đổi thủ công trong VSC, hãy đảm bảo các biến hệ thống đặc thù như `DIMSCALE` được quản lý đồng bộ trong Transaction. Đồng thời, luôn kiểm tra thuộc tính `LayerTableRecord.IsLocked` trước khi ghi đè dữ liệu lên layer hình học để tránh lỗi crash AutoCAD.